

BSc (Hons) in **Computer Science****SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing****Practical 01**

**Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

**Part 1: Code Analysis & PEAS Evaluation****Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

**Task 1.1: The Environment State (`_init_`)**

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P - performance  
E - Environment  
A - actuators  
S - sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
width
height
num_food
num_opponenets
custom_walls
agent_pos
food_positions
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

```
multi agent
the code creates self.opponents which means there are other moving opponents in the
game alongside the main agent. Since the agent is not alone in the game it is a multi
agent
```

## Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

```
percept sequence is the history of everything agent has sensed from the moment it was
turned on up to the current moment.
```

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

```
Sensor
```

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

partially observable

what agent can see - it's position, opponent's position, total count of remaining food  
what agent cannot see - food position, exact wall position

### Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance measure

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Agent's success is determined by what it has accomplished in the external environment like avoiding walls and collecting food rather than processing a complex code while doing nothing inside the game

## Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

## Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

if we intentionally hide the traps from the agent it cannot see the full state of the world. environment's observability classification becomes partially observable.

## Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos)` in `self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Rational agents want to maximize the score. By adding 'smells\_toxin' we feed the agent where the traps are located in it's percept sequence and it can act smart to avoid them and preserving its score and acting logically to get the best outcome.

## Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action` , check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid` , iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

If a vacuum cleaning agent is rewarded purely for cleaning up dirt, it learns to clean dirt, intentionally dump the dirt back onto the floor, and clean it up again repeatedly.

This allows the agent to score endless points without keeping the room clean. It shows why we must design performance metrics carefully so agents cannot exploit them.

Finalize and Lock Lab 01 Analytical Evaluation

**Incomplete: 10/11 questions answered. Please provide detailed responses for all questions.**



**FACULTY OF COMPUTING**